TP Noté — Algorithmes Génétiques

Problème du Voyageur de Commerce (TSP)

1. Contexte et objectif

Le Problème du Voyageur de Commerce (TSP — Travelling Salesman Problem) est l'un des problèmes d'optimisation combinatoire les plus célèbres en informatique. Étant donné un ensemble de villes et les distances entre elles, il s'agit de trouver le plus court chemin qui visite chaque ville exactement une fois et revient au point de départ.

Ce problème étant NP-difficile, il n'existe pas d'algorithme polynomial permettant de garantir la solution optimale pour de grandes instances. Les algorithmes génétiques constituent une approche heuristique efficace pour obtenir de bonnes solutions en temps raisonnable.

Objectif du TP : implémenter un algorithme génétique en Python pour résoudre le TSP, le soumettre à un serveur de compétition, et tenter de vous hisser au sommet du classement !

2. Les instances du concours

Six instances de difficulté croissante sont disponibles sur le serveur. Toutes utilisent des coordonnées géographiques réelles en France métropolitaine. La distance entre deux villes est la distance euclidienne entre leurs coordonnées longitude/latitude.

Identifiant	Nom	Nombre de villes
`regions`	Tiny — Tour de France des régions	**13**
`random_50`	Small — Livraison express	**50**
`prefectures`	Medium — Tournée des préfectures	**94**
`random_100`	Hard — Road trip improvisé	**100**
`random_200`	Extreme — Le facteur est fou	**200**
`random_500`	Nightmare — Cauchemar du livreur	**500**

3. Interface avec le serveur

Le serveur (`https://tsp-sra0.onrender.com/`) expose une API REST. Vous devez implémenter vous-même le client Python permettant d'interagir avec lui. L'URL de base du serveur vous sera communiquée au début de la séance.

Votre mission

Implémentez en Python les fonctions nécessaires pour : récupérer une instance, calculer la distance d'un tour, et soumettre votre meilleure solution.

La librairie `requests` est recommandée pour effectuer les appels HTTP.

3.1 Lister les instances disponibles

```
GET /instances

Réponse :
{
  "regions":     { "name": "Tiny — Tour de France des régions", "n_cities": 13
},
  "random_50":   { "name": "Small — Livraison express",          "n_cities": 50
},
  "prefectures": { "name": "Medium — Tournée des préfectures",   "n_cities": 94
},
  ...
}
```

3.2 Récupérer une instance

```
GET /instances/{instance_id}

Exemple : GET /instances/regions

Réponse :
{
  "instance_id": "regions",
  "name": "Tiny — Tour de France des régions",
  "cities": [
    [2.347, 48.859],
    [4.835, 45.748],
    ...
  ]
}
```

Chaque ville est représentée par ses coordonnées `[longitude, latitude]`. Les indices vont de 0 à n−1.

3.3 Soumettre une solution

```
POST /submit

Format attendu (exemple) :
{
  "student_id": "alice_martin",
  "instance_id": "regions",
  "tour": [0, 5, 2, 7, 1, 3, 4, 6, 8, 12, 11, 9, 10]
}

Réponse (succès) :
{
  "status":   "accepted",
  "distance": 1842.57,
```

```
  "improved": true,
  "in_top5":  true,
  "rank":     2
}

Réponse (erreur) :
{
  "detail": "La solution doit contenir chaque indice exactement une fois"
}
```

Le champ `tour` attendu est une liste d'indices de villes dans l'ordre de visite, par exemple `[0, 5, 2, 7, 1, ...]`. Chaque indice doit apparaître exactement une fois.

Codes HTTP retournés : `200` si la solution est acceptée, `400` si la solution est invalide (message d'erreur explicite dans `detail`).

Si la solution est acceptée, le serveur renvoie différentes informations :

- `distance` : la distance totale du tour, en kilomètres ;
- `improved` : `true` si cette solution améliore la meilleure solution soumise pour cette instance ; `false` sinon ;
- `in_top_5` : `true` si cette solution fait entrer dans le top 5 pour cette instance ; `false` sinon ;
- `rank` : rang dans le top 5, si `in_top_5 = true`

3.4 Calcul de la distance

Pour simplifier, la distance entre deux villes est la distance à vol d'oiseau. Les points étant donnés par leur longitude et latitude, et étant situés sur une sphère, la distance sera calculée avec la **formule de Haversine** et exprimée en kilomètres :

```
a = sin²(Δlat/2) + cos(lat_A) × cos(lat_B) × sin²(Δlon/2)
d(A, B) = 2 × R × arcsin(√a)

avec R = 6371 km (rayon moyen de la Terre),
     Δlat = lat_B − lat_A  et  Δlon = lon_B − lon_A  (en radians)
```

Note : cette distance est à vol d'oiseau en ligne droite. Elle peut donc traverser des frontières ou la mer.

Le serveur recalcule systématiquement la distance de votre tour côté serveur avec cette même formule. C'est cette valeur qui fait foi pour le classement.

4. Travail demandé

4.1 Implémentation de l'algorithme génétique

Vous devez implémenter un algorithme génétique en Python permettant de résoudre le TSP. Votre implémentation doit comporter au minimum les composants suivants :

1. Une représentation des individus adaptée au TSP.
2. Une initialisation de la population (au moins partiellement aléatoire).
3. Un opérateur de sélection des parents.
4. Un opérateur de croisement préservant la validité des permutations.
5. Un opérateur de mutation.
6. Un critère d'arrêt (nombre de générations, temps écoulé, convergence...).

Liberté totale : le choix des opérateurs, des paramètres (taille de population, taux de mutation, etc.) et des stratégies d'optimisation vous est entièrement laissé. Soyez créatifs !

4.2 Expérimentation et soumission

Une fois votre algorithme implémenté, testez d'abord sur les petites instances (`regions`, `random_50`) pour valider votre implémentation.

Soumettez vos meilleures solutions au fur et à mesure — vous pouvez soumettre autant de fois que vous le souhaitez, seul votre meilleur score est conservé.

Montez progressivement en difficulté vers les instances plus grandes.

Observez le leaderboard projeté en classe pour suivre le classement en temps réel !

4.3 Compte-rendu

À rendre en fin de séance (dépôt Moodle) :

- Votre fichier Python contenant l'algorithme génétique, commenté.
- Un court compte-rendu décrivant vos choix d'implémentation : opérateurs utilisés, paramètres retenus, et les améliorations que vous avez essayées (format Markdown exporté en PDF accepté)

5. Évaluation

Critère	Points
Implémentation correcte d'un algorithme génétique (composants obligatoires présents)	**8**
Qualité des solutions soumises (distance obtenue sur les instances)	**6**
Qualité du compte-rendu (clarté, pertinence des choix, analyse)	**4**
Bonus : top 3 sur au moins une instance	**+2**